CSC490 Assignment A5

Chris Jiang (Student ID: 1008109574) | Yihe Li (Student ID: 1009802331)
Qianwen Wang (Student ID: 1009985358) | Seonghyun Ban (Student ID: 1005127738)

March 28, 2026

1 Part 1

Objective

The goal of Part 1 was to profile important backend functions in the transaction-normalization path, identify the main performance bottlenecks, and make targeted improvements based on the profiling results. Because normalization is applied to every incoming transaction-like message, it is a good place to look for performance gains. Even small inefficiencies can accumulate when the same operations run repeatedly across many inputs.

Functions Profiled

Using Python's built-in `profile` module, I profiled the following five functions:

1. `NormalizationService.normalize_text`
2. `NormalizationService.extract_amount_mentions`
3. `NormalizationService.extract_date_mentions`
4. `NormalizationService.extract_party_mentions`
5. `NormalizationService.normalize`

The profiling script used for this part is:

- `autobook/scripts/profile_a5_backend.py`

It generates per-function profiling artifacts in

- `autobook/a5_artifacts/profiling/`

The generated files are:

- `autobook/a5_artifacts/profiling/normalize_text.txt`
- `autobook/a5_artifacts/profiling/extract_amount_mentions.txt`
- `autobook/a5_artifacts/profiling/extract_date_mentions.txt`
- `autobook/a5_artifacts/profiling/extract_party_mentions.txt`
- `autobook/a5_artifacts/profiling/normalize.txt`
- `autobook/a5_artifacts/profiling/summary.md`

Why These Functions Were Chosen

These five functions represent the core normalization workflow.

`normalize_text` is part of the hot path because it runs on every transaction-like message. Although its work is simple, it is called very frequently.

`extract_amount_mentions` is important because amount extraction is one of the most central parts of transaction normalization. It must also avoid false positives from dates, invoice numbers, and repeated

numeric tokens.

`extract_date_mentions` matters because transaction descriptions often include dates in multiple formats, especially in upload-derived or semi-structured text.

`extract_party_mentions` is important because vendor or counterparty detection supports downstream categorization and bookkeeping context.

`normalize` is the end-to-end orchestration method. It calls the helper functions, applies fallback and precedence logic, and constructs the normalized transaction candidate used by downstream services.

Profiling Setup

To make the profiling realistic, the script generates **3,000 representative transaction messages**. These messages vary across several dimensions, including:

- vendor names
- dates in different formats
- quantities
- invoice numbers
- manual-input source types
- upload-like source types
- mixed structured and unstructured text

Each target function is executed over this dataset, and the script records both detailed profiling output and wall-clock runtime.

Profiling Results

The summary in `autobook/a5_artifacts/profiling/summary.md` reported the following wall-clock times:

Function	Wall-clock time
`normalize`	543.68 ms
`extract_party_mentions`	117.88 ms
`extract_amount_mentions`	111.35 ms
`extract_date_mentions`	73.38 ms
`normalize_text`	22.98 ms

Interpretation of Results

The slowest function was `normalize`, which is expected because it performs the full end-to-end normalization workflow and combines the cost of the helper functions.

Among the helpers, the two most expensive were `extract_party_mentions` and `extract_amount_mentions`. This matches the structure of the module, since both functions rely heavily on repeated regex matching over free-form text and must distinguish between similar-looking token patterns.

For example, an amount-like token may actually be part of a date or invoice number, while a party-like substring may actually be a generic accounting term rather than a real vendor. This makes both functions logically complex and computationally heavier than simpler utilities.

`extract_date_mentions` was also nontrivial, reflecting the cost of scanning for multiple date patterns and handling malformed values.

`normalize_text` was the fastest function, but it still remained worth optimizing because it runs on every input.

Overall, the profiling results suggested two main conclusions:

1. The main hot spots are concentrated in the regex-heavy extraction helpers.
2. The full `normalize()` path should also be checked for duplicated work across helper calls.

Improvements Made

Based on the profiling results, I made three focused optimizations in:

- `autobook/backend/services/shared/normalization.py`

These changes were selected because they directly address the measured hot spots.

1. Precompiled regular expressions

The original implementation repeatedly called `re.finditer`, `re.search`, and `re.sub` with raw pattern strings. That means the same regex patterns could be compiled many times during batch processing.

To reduce this overhead, I moved the main patterns into module-level precompiled constants:

- `AMOUNT_REGEXES`
- `DATE_REGEXES`
- `DATE_SCRUB_REGEX`
- `NUMERIC_TOKEN_REGEX`
- `YEAR_TOKEN_REGEX`
- `PARTY_REGEXES`
- `QUANTITY_REGEX`

This optimization is especially relevant for `extract_amount_mentions`, `extract_date_mentions`, and `extract_party_mentions`, since those functions repeatedly scan text using the same pattern families.

2. Faster whitespace normalization

I updated `normalize_text` to use:

```
" ".join(text.lower().split())
```

instead of a regex-based whitespace substitution.

For this specific task, split-and-join is a better fit. It lowercases the text, collapses arbitrary internal whitespace, and rebuilds a clean string without invoking regex machinery.

Although `normalize_text` was not the largest bottleneck, it is a high-frequency function, so reducing its per-call overhead is still useful.

3. Reuse of precomputed date mentions

While profiling the full `normalize()` path, I noticed that `date_mentions` were already being computed, but downstream transaction-date extraction could still reparse the same description text.

To remove that duplicated work, I modified `extract_transaction_date()` so that it can accept precomputed date mentions and use them before reparsing raw text.

This optimization is important because it removes redundant scanning from the end-to-end normalization workflow.

Outcome

Part 1 produced two useful outcomes.

First, the profiling artifacts clearly identify where time is being spent in the normalization path. The main cost is concentrated in the regex-heavy extraction helpers and in the full `normalize()` orchestration function.

Second, the implemented optimizations directly target those costs. Precompiled regexes reduce repeated setup overhead, split/join simplifies a hot-path text-normalization step, and reuse of precomputed date mentions avoids redundant parsing.

Function	**Before (ms)**	**After (ms)**	**Change (ms)**
`normalize`	543.68	521.23	-22.45
`extract_party_mentions`	117.88	111.50	-6.38
`extract_amount_mentions`	111.35	107.94	-3.41
`extract_date_mentions`	73.38	69.95	-3.43
`normalize_text`	22.98	20.03	-2.95

Table 1: Wall-clock times before and after the Part 1 optimizations.

2 Part Two: Showing Code Coverage

2.1 Setup

We created a GitHub Actions workflow (`.github/workflows/test.yml`) that runs automatically on every pull request targeting the `main` branch. The workflow uses the following tools:

- **pytest-cov** — configured in `pyproject.toml` with branch coverage enabled (`branch = true`), generating terminal, XML, and HTML reports on every run.
- **MishaKav/pytest-coverage-comment** — a GitHub Action that parses the coverage XML and JUnit XML reports, then posts a rich coverage table as a sticky comment on the PR. The comment includes per-file coverage, clickable file links, pass/fail counts, and execution time. It updates in place on each push rather than creating duplicate comments.
- **shields.io endpoint badge** — the workflow extracts the coverage percentage from the XML report, writes a JSON badge file to `.github/badges/coverage.json`, and commits it to the source branch. The README references this file via a shields.io endpoint URL, rendering a live coverage badge.

2.2 How It Works

1. Open a PR from `autobook` to `main`.
2. GitHub Actions triggers `test.yml`, which installs dependencies via `uv`, runs `pytest` with coverage, and generates XML + HTML reports.
3. The workflow posts a sticky coverage comment on the PR with the full per-file breakdown.
4. The workflow commits `coverage.json` to the source branch, updating the README badge.
5. The HTML coverage report is uploaded as a downloadable artifact on the workflow run.

2.3 Screenshots

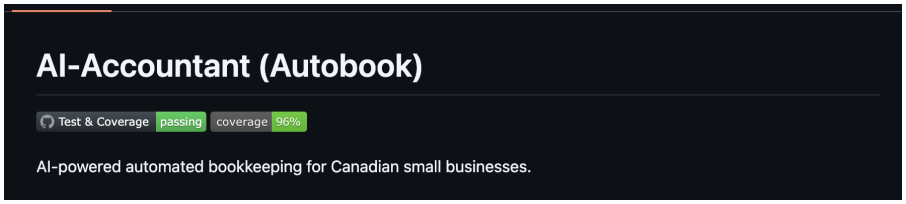

Figure 1: Coverage badge in readme.md.

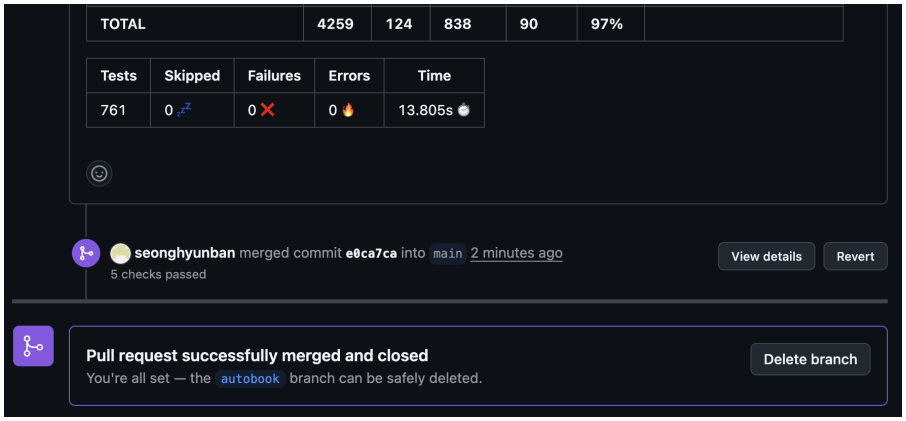

Figure 2: Coverage table posted automatically on the pull request.

3 Part Three: Test Coverage Breadth

3.1 Coverage Scope

The test suite measures **4,259 statements** across **166 Python files** in the backend, covering all layers of the system: API routes, authentication, database models and DAOs, queue operations, reporting, and all 8 pipeline services (normalizer, precedent, ML inference, LLM agent, resolution, posting, flywheel, and shared utilities).

3.1.1 Omissions

Only two categories are omitted from coverage measurement, both structurally untestable:

Omitted Path	Justification
`services/**/__main__.py`	Infinite `while True` polling loops for local development. No input, no output, no return — cannot be entered or exited in a test.
`api/routes/events.py`	Server-Sent Events async generator over live Redis pub/sub. Yields indefinitely until client disconnect — requires real infrastructure and has no testable termination.

3.2 Mocking Strategy

External dependencies are mocked at the boundary to isolate business logic:

- **AWS services**: `boto3` mocked for SageMaker `InvokeEndpoint`, SQS via `moto`, Secrets Manager via `moto`.
- **LLM providers**: `langchain_aws.ChatBedrockConverse` mocked via `sys.modules` stubs. Agent nodes tested by mocking `get_llm()` to return a fake structured output chain.
- **Vector database**: `qdrant_client.QdrantClient` and `vectordb.embeddings.embed_text` mocked. Collection initialization tested with mock client.
- **ML models**: `torch` and `transformers` mocked via `sys.modules` stubs. DeBERTa classifier and NER provider logic tested through mock model pipelines. BIO span parsing tested as pure Python.
- **Database**: SQLite in-memory (`DATABASE_URL=sqlite:///:memory:`) replaces PostgreSQL. `set_current_user_context` (PostgreSQL-only RLS function) patched as no-op.
- **Redis**: `fakeredis` for cache and pub/sub tests.

Business logic is tested directly without mocks: normalization rules, precedent scoring, heuristic classification, accounting rules, journal entry validation, financial statement generation, routing logic, and the fix-loop scheduler.

3.3 Result

Metric	Value
Total tests	761
Failures	0
Total statements measured	4,259
Statements covered	4,135
Statements missing	124
Branch coverage enabled	Yes
Coverage	**96%**

3.4 Grading

$$96\% \times 0.20 = \mathbf{19.2} \ / \ \mathbf{20\ marks}$$

4 Part 4

Module Selected

For Part 4, I selected:

- `autobook/backend/services/shared/normalization.py`

I chose this module because it is one of the most practically complex modules in the backend and one whose correctness directly affects downstream system behavior.

Its responsibilities include:

- normalizing heterogeneous text input
- inferring amounts from both explicit fields and free-form descriptions
- distinguishing dates from other numeric tokens
- identifying counterparties while filtering generic non-party words
- extracting quantities without confusing them with currencies or invoice references
- handling uploads and incomplete messages safely
- preserving explicit upstream values when they should take precedence

This mixture of parsing, normalization, precedence logic, ambiguity handling, and fallback behavior makes it a strong candidate for deep unit testing.

Test File

The unit tests added for this part are located in:

- `autobook/tests/services/shared/test_normalization.py`

This file contains **15 unit tests**, which exceeds the assignment requirement of at least 10 tests.

Testing Strategy

My goal was not only to meet the minimum number of tests but also to cover the behaviors most likely to affect downstream accounting quality.

I structured the tests around three kinds of behavior:

1. **Canonicalization and basic normalization**
   Examples: source alias mapping, whitespace normalization, and text canonicalization.
2. **Extraction helper behavior**
   Examples: amount extraction, date extraction, party extraction, and quantity extraction.
3. **End-to-end normalization behavior**
   Examples: fallback defaults, explicit-value precedence, reuse of extracted mentions, and standard successful normalization flows.

This approach helps cover both local helper logic and integrated behavior in `normalize()`.

Edge Cases and Rationale

The normalization module includes a module-level comment linking the main edge-case categories to the test suite and explaining why these tests were chosen. For readability, I group the most important tests into helper-level tests and end-to-end / precedence tests.

Helper and extraction tests

Test	Edge case / scenario	Why this matters
`test_canonicalize_source_maps_known_aliases`	source aliases such as `manual` and `CSV`	prevents source-specific logic from breaking due to inconsistent labels
`test_normalize_text_collapses_internal_whitespace`	repeated tabs, newlines, and spaces	ensures downstream matching sees a stable canonical string
`test_extract_amount_mentions_deduplicates_regex_and_numeric_matches`	same amount matched twice in different ways	avoids duplicate amount mentions and artificial ambiguity
`test_extract_amount_mentions_ignores_date_tokens`	dates containing numbers near amounts	prevents date fragments from being mistaken for monetary values
`test_extract_amount_prefers_valid_explicit_amount`	valid upstream explicit amount override	preserves precedence of trusted structured input
`test_extract_amount_invalid_explicit_falls_back_to_single_mention`	invalid explicit amount with recoverable text input	verifies graceful fallback instead of unnecessary failure
`test_extract_amount_returns_not_confident_for_multiple_mentions`	multiple plausible amount candidates	confirms that ambiguity is represented explicitly
`test_extract_date_mentions_normalizes_slash_dates_and_keeps_invalid_raw_value`	mixture of valid and invalid date strings	checks both normalization and robust malformed-input handling
`test_extract_party_mentions_filters_non_party_tokens`	generic words such as `cash` in a party-like position	prevents false vendor extraction
`test_extract_quantity_mentions_ignores_currency_units_and_deduplicates`	quantities mixed with `CAD`, invoice numbers, and repeated items	prevents misclassification of currencies or metadata as quantities

Precedence and end-to-end tests

Test	Edge case / scenario	Why this matters
`test_extract_counterparty_prefers_explicit_message_value`	explicit counterparty conflicts with parsed mention	verifies correct precedence rules
`test_extract_transaction_date_prefers_existing_message_value`	explicit transaction date already present	protects trusted upstream structured values
`test_extract_transaction_date_uses_precomputed_mentions_before_reparsing`	reuse of extracted date data	validates the optimization and guards against redundant parsing
`test_normalize_uses_filename_fallback_defaults_and_single_mentions`	upload-driven message with filename but missing normal input text	checks safe fallback behavior for realistic upload workflows
`test_normalize_builds_mentions_for_standard_manual_text`	standard positive end-to-end normalization flow	validates the main successful path

Why These Cases Matter

These tests were chosen because normalization errors are often subtle but high-impact.

For example, duplicate amount extraction may create false ambiguity. Date fragments may be misclassified as transaction amounts. Generic words may be mistaken for counterparts. Quantity extraction may confuse currency markers or invoice identifiers for item counts. Upload-derived records may also fail to normalize safely if fallback behavior is not correct.

These are realistic failure modes for a normalization layer that processes heterogeneous free-form input. For that reason, the test suite emphasizes both correct extraction and safe handling of malformed or ambiguous cases.

Positive and Negative Tests

The assignment requires both positive and negative tests, and this suite includes both.

Positive tests

The positive tests verify correct behavior on valid or clean inputs. These include:

- standard manual-text normalization
- valid explicit amount handling
- correct source alias mapping
- correct extraction of party, amount, and date from clean input
- correct end-to-end output construction for common cases

These tests confirm that the module behaves as intended on the normal usage path.

Negative tests

The negative tests verify graceful handling of ambiguous, conflicting, or incomplete input. These include:

- ambiguous multiple amount mentions
- malformed explicit amount values
- invalid slash-format date strings
- non-party tokens in vendor-like positions
- quantity patterns near currencies and invoice references
- upload messages missing standard text fields
- conflicts between explicit and parsed values

These tests are especially important because normalization code often fails silently. The risk is not necessarily that the system crashes, but that it produces plausible-looking structured data that is wrong. The negative tests therefore check robust fallback and ambiguity-aware behavior.

Depth of Coverage

I believe this suite demonstrates depth for three reasons.

First, it tests both helper-level behavior and end-to-end behavior. Some tests target extraction helpers directly, while others validate the integrated behavior of `normalize()`.

Second, it tests precedence and fallback logic. Several tests are about correct decision-making when multiple information sources are available, not just whether extraction succeeds.

Third, it explicitly targets ambiguity and misclassification. In several cases, the correct behavior is to deduplicate, reject misleading matches, preserve raw malformed values where appropriate, or represent uncertainty rather than overcommitting.

The test run passed with **15 tests**.

5 Part 5

Link to video: Demo

5.1 Video summary

Bug 1: Label Leakage in the ML Training Pipeline Gold labels were inadvertently included in training inputs, leading to artificially inflated performance.

Problem Description The sequence-classifier training pipeline was leaking gold-label information into the model inputs. Specifically:

- `bank_category` examples included the true `intent_label`.
- `cca_class_match` examples included both `intent_label` and `asset_name`.

This issue was identified during training process by monitoring the training loss curve.

Root Cause Analysis The issue originated from the data construction and training pipeline:

- `ml_workspace/training/hf_dataset.py` constructed inputs with label-derived fields.
- `ml_workspace/training/train_deberta_sequence.py` trained multiple tasks jointly without isolating label dependencies.

This resulted in implicit leakage of target variables into model features.

Fix Implemented

- Removed leaked-label sequence tasks from the training pipeline.
- Restricted the sequence classifier to `intent_label` only.
- Separated entity extraction into an independent model.

This restructuring aligns the training pipeline with the runtime contract.

Lesson Learned Check training setup carefully before training and be cautious with label leakage that might cause model cheating and getting invalid results.

Regression Testing Validation is performed using:

- `evaluate_saved_models.py`
- `classifier_run_summary.json`
- `test_metrics.json`

These confirm that only `intent_label` is trained and evaluated.

5.2 Bug 2: Hybrid Inference Overwriting Strong Normalizer Signals

Weak NER outputs were overriding stronger deterministic signals during entity merging.

Problem Description In the hybrid inference layer, NER outputs were overriding higher-confidence structured signals from the normalizer, particularly:

- `vendor`
- `mentioned_date`

This degraded final output quality despite correct upstream extraction.
This issue was found during the check of training results.

Root Cause Analysis Inspection of:

- `autobook/backend/services/ml_inference/logic.py`

revealed that:

- The `extract_entities` merge logic applied NER outputs whenever non-empty.
- No priority or confidence hierarchy was enforced between sources.

Fix Implemented

- Introduced helper methods:
  - `_preferred_vendor`
  - `_preferred_mentioned_date`
- Enforced priority order:
  1. Normalizer signals (`counterparty`, `party_mentions`, `date_mentions`)
  2. Fallback to NER only when necessary
- Updated merge logic to restore preferred values before returning results.

Lesson Learned In hybrid systems, learned components should not blindly override deterministic outputs. Explicit precedence or confidence-based merging is required.

Regression Testing A targeted test was added:

- `test_hybrid_service_prefers_normalizer_vendor_and_date_over_ner`

Test setup:

- Fake NER outputs: `"Coffee"`, `"-03-14"`
- Expected outputs:
  - Vendor: `"Pilot Coffee Roasters"`
  - Date: `"2026-03-14"`

This ensures that normalizer-derived values are preserved over weaker ML predictions.